

# 手写数字识别

## 神经网络初探

WyEhNf

上海交通大学 2025 级 ACM 班

2025 年 12 月 17 日



# 计算机如何识别图像？

# 人眼 vs 计算机视觉

## 人类如何看图？

- 能够直观地看到数字
- 快速识别和理解图像内容



# 人眼 vs 计算机视觉

## 人类如何看图？

- 能够直观地看到数字
- 快速识别和理解图像内容

## 计算机看图的困难

- 只能处理 0 和 1 的二进制数据
- 需要把视觉信息转换为数学形式



# 人眼 vs 计算机视觉

## 人类如何看图？

- 能够直观地看到数字
- 快速识别和理解图像内容

## 计算机看图的困难

- 只能处理 0 和 1 的二进制数据
- 需要把视觉信息转换为数学形式

## 核心挑战

如何让计算机“理解”图像内容



# 数字图像的基本概念

## 像素 (Pixel)

- 图像的基本单位
- 像素数量决定图像分辨率



# 数字图像的基本概念

## 像素 (Pixel)

- 图像的基本单位
- 像素数量决定图像分辨率

## 灰度图像

每个像素用 0-255 表示亮度

- 0: 纯黑色
- 255: 纯白色



# 彩色图像与 MNIST 数据

## 彩色图像 (RGB)

- 每个像素由 R、G、B 三原色表示
- 每个通道 0-255, 共 1670 万种颜色





# 彩色图像与 MNIST 数据

## 彩色图像 (RGB)

- 每个像素由 R、G、B 三原色表示
- 每个通道 0-255, 共 1670 万种颜色

## MNIST 手写数字

- $28 \times 28$  像素的灰度图
- 总共 784 个像素点
- 每个像素值范围: 0-255



# 数字矩阵表示

## 矩阵表示

每张 MNIST 图像对应  $28 \times 28$  数字矩阵

- 矩阵元素：像素灰度值
- 数值大：笔迹所在（亮）
- 数值小：背景区域（暗）



# 数字矩阵表示

## 矩阵表示

每张 MNIST 图像对应  $28 \times 28$  数字矩阵

- 矩阵元素：像素灰度值
- 数值大：笔迹所在（亮）
- 数值小：背景区域（暗）

## 示例

行 1:  $[0, 0, 0, \dots, 0, 0, 0]$

行 2:  $[0, 0, 0, \dots, 234, 0, 0]$

行 3:  $[0, 0, 120, \dots, 253, 120, 0]$



# 图像数据处理

## 向量化 (展平)

- 将  $28 \times 28$  矩阵拉平为一维数组
- 得到长度为 784 的一维向量



# 图像数据处理

## 向量化 (展平)

- 将  $28 \times 28$  矩阵拉平为一维数组
- 得到长度为 784 的一维向量

## 数据归一化

- 像素值除以 255
- 归一化到  $[0, 1]$  区间
- 示例:  $127 \rightarrow 0.498$



# 识别的挑战

## 手写数字的多样性

- 同一个数字有不同写法
- 笔画粗细、倾斜角度各异
- 类似数字容易混淆



# 识别的挑战

## 手写数字的多样性

- 同一个数字有不同写法
- 笔画粗细、倾斜角度各异
- 类似数字容易混淆

## 特征

能够区分不同类别的关键信息



# 特征提取

## 好特征的特点

- 对同一类别稳定
- 对不同类别有区分性
- 对噪声鲁棒





# 特征提取

## 好特征的特点

- 对同一类别稳定
- 对不同类别有区分性
- 对噪声鲁棒

## Examples

- 数字“1”： 竖直笔画
- 数字“8”： 两个闭合区域
- 数字“0”： 一个闭合圆形



# Solution: 从人眼到计算机的转换

## ① 图像数字化：将视觉信息转换为数学表示



## Solution: 从人眼到计算机的转换

- ① **图像数字化**: 将视觉信息转换为数学表示
- ② **数据预处理**: 向量化、归一化便于处理



## Solution: 从人眼到计算机的转换

- ① **图像数字化**: 将视觉信息转换为数学表示
- ② **数据预处理**: 向量化、归一化便于处理
- ③ **特征提取**: 识别关键区分信息



## Solution: 从人眼到计算机的转换

- ① **图像数字化**: 将视觉信息转换为数学表示
- ② **数据预处理**: 向量化、归一化便于处理
- ③ **特征提取**: 识别关键区分信息
- ④ **机器学习**: 自动学习模式实现识别



# 本例的形式化

## 特征空间

特征空间记为  $X \subseteq \mathbb{R}^d$ , 本例中  $d = 784$  ( $28 \times 28$  图像展平)



# 本例的形式化

## 特征空间

特征空间记为  $X \subseteq \mathbb{R}^d$ , 本例中  $d = 784$  ( $28 \times 28$  图像展平)

## 标签空间

标签空间记为  $Y = \{0, 1, \dots, 9\}$ , 共 10 个数字类别



# 本例的形式化

## 特征空间

特征空间记为  $X \subseteq \mathbb{R}^d$ , 本例中  $d = 784$  ( $28 \times 28$  图像展平)

## 标签空间

标签空间记为  $Y = \{0, 1, \dots, 9\}$ , 共 10 个数字类别

## 训练集

$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ , 用于模型训练







# 机器学习与神经网络

# 机器学习任务的分类 (按监督信息)

- **监督学习**: 学习从  $x$  到  $y$  的映射



# 机器学习任务的分类 (按监督信息)

- **监督学习**: 学习从  $x$  到  $y$  的映射
- **非监督学习**: 挖掘数据内部结构



## 机器学习任务的分类 (按监督信息)

- **监督学习**: 学习从  $x$  到  $y$  的映射
- **非监督学习**: 挖掘数据内部结构
- **半监督学习**: 标注 + 未标注样本结合



# 机器学习任务的分类 (按监督信息)

- **监督学习**: 学习从  $x$  到  $y$  的映射
- **非监督学习**: 挖掘数据内部结构
- **半监督学习**: 标注 + 未标注样本结合
- **强化学习**: 环境交互学习



# 机器学习任务的分类 (按输出类型)

## 分类 (classification)

$Y$  为有限类别集合, 如手写数字识别 (0-9)



# 机器学习任务的分类 (按输出类型)

## 分类 (classification)

$Y$  为有限类别集合, 如手写数字识别 (0-9)

## 回归 (regression)

$Y$  为实数或向量, 适用于连续数值预测



# 机器学习任务的分类 (按输出类型)

## 分类 (classification)

$Y$  为有限类别集合, 如手写数字识别 (0-9)

## 回归 (regression)

$Y$  为实数或向量, 适用于连续数值预测

## 本例定位

典型的监督式多分类问题





# 分类的概率视角

## 生成式模型

建模  $P(X|Y)$  与  $P(Y)$ , 使用贝叶斯公式



# 分类的概率视角

## 生成式模型

建模  $P(X|Y)$  与  $P(Y)$ , 使用贝叶斯公式

## 判别式模型

直接建模  $P(Y|X)$  或决策函数



# 分类的概率视角

## 生成式模型

建模  $P(X|Y)$  与  $P(Y)$ , 使用贝叶斯公式

## 判别式模型

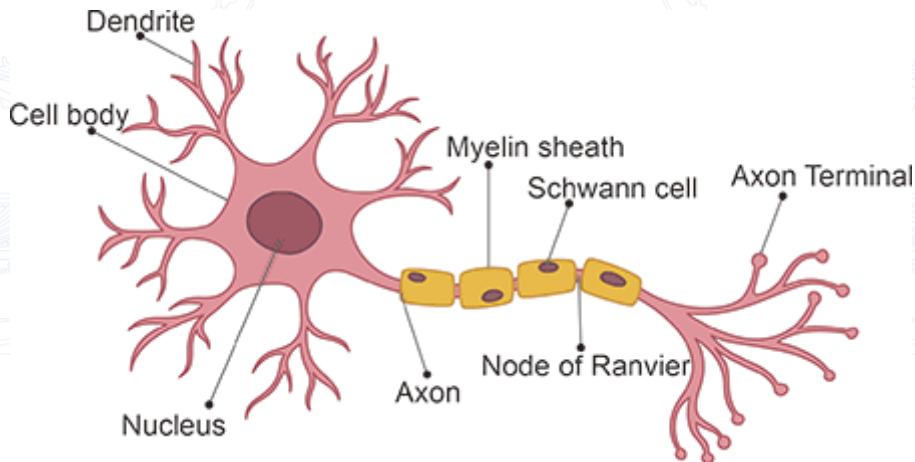
直接建模  $P(Y|X)$  或决策函数

## 神经网络分类器

- 通常为判别式模型
- 直接学习从输入  $x$  到类别  $y$  的映射



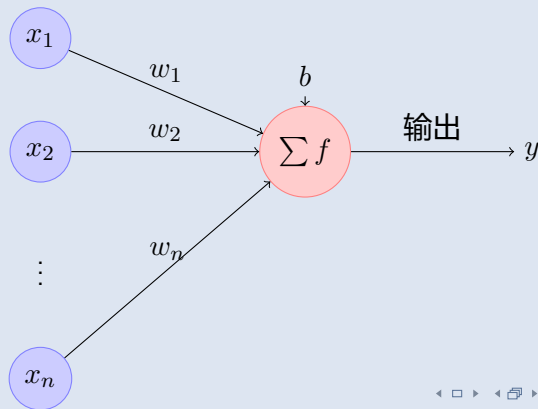
# 生物中的神经元



# 神经网络的基本原理

## 人工神经元的数学表达

### 人工神经元模型



# 仿生基础

## 计算公式

$$y = f \left( \sum_{i=1}^n w_i x_i + b \right)$$

- $x_i$ : 输入信号
- $w_i$ : 连接权重
- $b$ : 偏置项
- $f$ : 激活函数



# 仿生基础

## 计算公式

$$y = f \left( \sum_{i=1}^n w_i x_i + b \right)$$

- $x_i$ : 输入信号
- $w_i$ : 连接权重
- $b$ : 偏置项
- $f$ : 激活函数

## 仿生对应

- 输入 → 树突信号
- 权重 → 突触强度
- 求和 → 信号整合
- 激活 → 放电阈值



# 激活函数

常见的激活函数有两种：





# 激活函数

常见的激活函数有两种：

## Relu 函数

$$f(x) = x[x > 0]$$



# 激活函数

常见的激活函数有两种：

## Relu 函数

$$f(x) = x[x > 0]$$

## Sigmoid 函数

$$f(x) = \frac{1}{1 + \exp(-x)}$$



# 激活函数

常见的激活函数有两种：

## Relu 函数

$$f(x) = x[x > 0]$$

## Sigmoid 函数

$$f(x) = \frac{1}{1 + \exp(-x)}$$

激活函数的目的是将卷积这一线性操作非线性化。



# 网络连接方式

## 全连接结构

- 输入层与隐藏层全连接
- 隐藏层与输出层全连接



# 网络连接方式

## 全连接结构

- 输入层与隐藏层全连接
- 隐藏层与输出层全连接

## 信息传递

输入层 → 隐藏层 → 输出层



# 网络连接方式

## 全连接结构

- 输入层与隐藏层全连接
- 隐藏层与输出层全连接

## 信息传递

输入层 → 隐藏层 → 输出层

## 参数数量

总参数量：约 40 万个



# 前向传播步骤 1: 输入层 → 隐藏层

## 线性组合

$$\text{hidden\_layer\_input}[j] = \sum_{i=1}^{784} \text{input\_vector}[i] \times \text{weights\_input\_hidden}[i][j]$$



# 前向传播步骤 1: 输入层 → 隐藏层

## 线性组合

$$\text{hidden\_layer\_input}[j] = \sum_{i=1}^{784} \text{input\_vector}[i] \times \text{weights\_input\_hidden}[i][j]$$

## 激活

$$\text{hidden\_layer\_output}[j] = \sigma(\text{hidden\_layer\_input}[j])$$





## 前向传播步骤 2: 隐藏层 → 输出层

### 线性组合

$$\text{output\_layer\_input}[k] = \sum_{j=1}^{500} \text{hidden\_layer\_output}[j] \times \text{weights\_hidden\_output}[j][k]$$



## 前向传播步骤 2: 隐藏层 → 输出层

### 线性组合

$$\text{output\_layer\_input}[k] = \sum_{j=1}^{500} \text{hidden\_layer\_output}[j] \times \text{weights\_hidden\_output}[j][k]$$

### 激活与预测

$$\text{output\_layer\_output}[k] = \sigma(\text{output\_layer\_input}[k])$$

预测类别:  $\text{predicted\_class} = \arg \max_k \text{output\_layer\_output}[k]$



# 输出层梯度计算

## 输出层误差项

$$\delta_j = (\text{output}_j - \text{target}_j) \times \sigma'(\text{output}_j)$$



# 输出层梯度计算

## 输出层误差项

$$\delta_j = (\text{output}_j - \text{target}_j) \times \sigma'(\text{output}_j)$$

## 权重梯度

$$\frac{\partial L}{\partial \text{weights\_hidden\_output}[i][j]} = \delta_j \times \text{hidden\_layer\_output}[i]$$



# 隐藏层误差计算与权重更新

## 隐藏层误差

$$\delta_j^{\text{hidden}} = \left( \sum_{k=0}^9 \delta_k \times \text{weights\_hidden\_output}[j][k] \right) \times \sigma'(\text{hidden\_layer\_output}[j])$$



# 隐藏层误差计算与权重更新

## 隐藏层误差

$$\delta_j^{\text{hidden}} = \left( \sum_{k=0}^9 \delta_k \times \text{weights\_hidden\_output}[j][k] \right) \times \sigma'(\text{hidden\_layer\_output}[j])$$

## 权重更新规则

$$\text{weights} \leftarrow \text{weights} - \eta \times \frac{\partial L}{\partial \text{weights}}$$

学习率  $\eta = 0.2$



# 全连接层的训练效果

表: 训练准确率趋势 (总准确率: 90.26%)

| 累积任务数<br>(Tasks) | 分块准确率 (%)<br>(Chunk Acc.) |
|------------------|---------------------------|
| 100              | 8                         |
| 1000             | 50                        |
| 5000             | 86                        |
| 10000            | 91                        |
| 20000            | 92                        |
| 30000            | 88                        |
| 40000            | 82                        |
| 50000            | 92                        |
| 60000            | 98                        |



## 全连接层的训练效果

表: 训练准确率趋势 (总准确率: 90.26%)

| 累积任务数<br>(Tasks) | 分块准确率 (%)<br>(Chunk Acc.) |
|------------------|---------------------------|
| 100              | 8                         |
| 1000             | 50                        |
| 5000             | 86                        |
| 10000            | 91                        |
| 20000            | 92                        |
| 30000            | 88                        |
| 40000            | 82                        |
| 50000            | 92                        |
| 60000            | 98                        |

表: 测试准确率趋势 (总准确率: 94.15%)

| 累积样本数<br>(Tests) | 分块准确率 (%)<br>(Chunk Acc.) |
|------------------|---------------------------|
| 100              | 97                        |
| 1000             | 90                        |
| 2000             | 91                        |
| 3000             | 95                        |
| 4000             | 88                        |
| 5000             | 93                        |
| 7000             | 98                        |
| 9000             | 100                       |
| 10000            | 92                        |





# 为什么训练效果不佳?

- 破坏空间结构



# 为什么训练效果不佳?

- 破坏空间结构
- 不具备平移不变性



# 为什么训练效果不佳？

- 破坏空间结构
- 不具备平移不变性
- 参数爆炸，容易记录过多噪音的特征



## 直观的改进想法

- 二维的平面?



## 直观的改进想法

- 二维的平面?
- 平移不变性: 模块化?



# 直观的改进想法

- 二维的平面?
- 平移不变性: 模块化?
- 缩减参数: 减少连接?

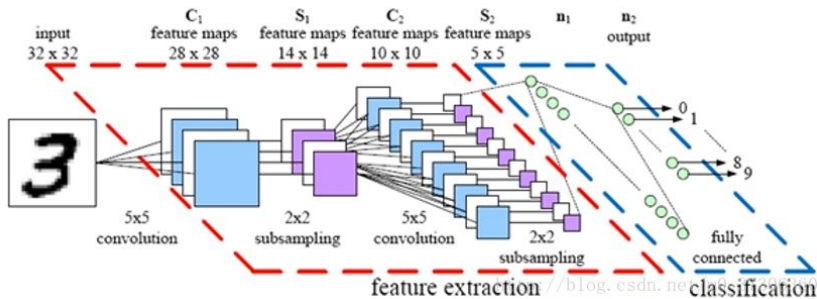




# 卷积神经网络 (CNN)

# CNN 模式图

## CNN





8

## 前向传播



# 卷积神经元

考虑一般的神经元，对于输入向量  $X$ ，其转换函数为：

$$f(x) = \sum x_i \cdot w_i + b$$



# 卷积神经元

考虑一般的神经元，对于输入向量  $X$ ，其转换函数为：

$$f(x) = \sum x_i \cdot w_i + b$$

而卷积神经元相当于将向量间的内积变为权值矩阵与输入向量/矩阵的卷积，即对于输入向量  $X$ ，其转换函数为：

$$f(x) = W^T * X + b$$



# 卷积神经元

考虑一般的神经元，对于输入向量  $X$ ，其转换函数为：

$$f(x) = \sum x_i \cdot w_i + b$$

而卷积神经元相当于将向量间的内积变为权值矩阵与输入向量/矩阵的卷积，即对于输入向量  $X$ ，其转换函数为：

$$f(x) = W^T * X + b$$

这对二维图像明显有这个更好的特征反映能力。



# 卷积神经元的优势

- 捕捉空间特征：卷积核是二维的



# 卷积神经元的优势

- 捕捉空间特征：卷积核是二维的
- 捕捉局部特征：平移不变性



# 卷积神经元的优势

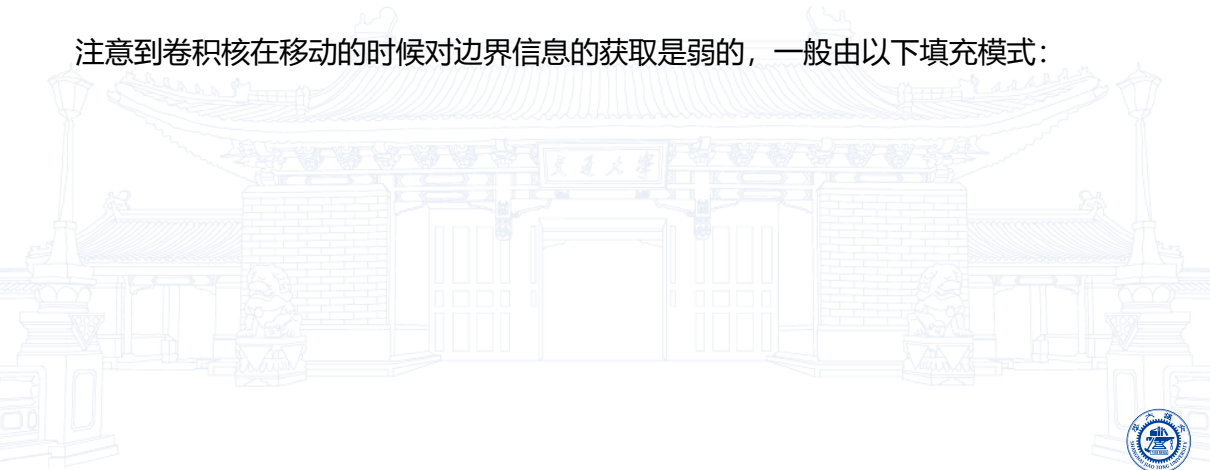
- 捕捉空间特征：卷积核是二维的
- 捕捉局部特征：平移不变性
- 模块化：卷积核的滑动保证了权值共享

卷积核的本质就是某一个特征的“模板”。每一个卷积核都可以检测一个特定的特征。



# 卷积核的移动与边界填充

注意到卷积核在移动的时候对边界信息的获取是弱的，一般由以下填充模式：





# 卷积核的移动与边界填充

注意到卷积核在移动的时候对边界信息的获取是弱的，一般由以下填充模式：

## Valid 模式

不进行额外填充， $r * c$  的矩阵经卷积后输出为  $(r - n + 1) * (c - m + 1)$  的矩阵。



## 卷积核的移动与边界填充

注意到卷积核在移动的时候对边界信息的获取是弱的，一般由以下填充模式：

### Valid 模式

不进行额外填充， $r * c$  的矩阵经卷积后输出为  $(r - n + 1) * (c - m + 1)$  的矩阵。

### Full 模式

充分填充使得卷积核中心经过输入矩阵每个位置， $r * c$  的矩阵经卷积后输出为  $(r + n - 1) * (c + m - 1)$  的矩阵。



## 卷积核的移动与边界填充

注意到卷积核在移动的时候对边界信息的获取是弱的，一般由以下填充模式：

### Valid 模式

不进行额外填充， $r * c$  的矩阵经卷积后输出为  $(r - n + 1) * (c - m + 1)$  的矩阵。

### Full 模式

充分填充使得卷积核中心经过输入矩阵每个位置， $r * c$  的矩阵经卷积后输出为  $(r + n - 1) * (c + m - 1)$  的矩阵。

### Same 模式

适当填充使得输出矩阵大小不变。



# 池化层

池化层的目的是降低进入后续层的数据规模，仅保留每个局部的整体特征。



# 池化层

池化层的目的是降低进入后续层的数据规模，仅保留每个局部的整体特征。  
池化窗口的移动和卷积核类似，但不存在填充的问题。



# 池化层

池化层的目的是降低进入后续层的数据规模，仅保留每个局部的整体特征。  
池化窗口的移动和卷积核类似，但不存在填充的问题。  
一般可以取窗口内平均或者  $\max/\min$ 。



# 全连接层

全连接层相当于对所得到的数据的总结，可以看作是一个收束的神经元，将数据数量约束到最终期望的规模，



# 全连接层

全连接层相当于对所得到的数据的总结，可以看作是一个收束的神经元，将数据数量约束到最终期望的规模，

特别的，全连接层的激活函数需要将数据转化为可供与标签比较的量。这个激活后的输出即为最终输出。





# 全连接层的激活函数

这里采用 softmax 函数，这个函数可以将一组数据归一化到  $(0, 1)$ ，且恰好能表示概率分布。



# 全连接层的激活函数

这里采用 softmax 函数，这个函数可以将一组数据归一化到 (0, 1)，且恰好能表示概率分布。

## Softmax 函数

$$\text{Softmax}(y_i) = \frac{\exp(y_i)}{\sum \exp(y_j)}$$



# 误差评估

每次评估误差后根据误差修正卷积核的参量。



# 误差评估

每次评估误差后根据误差修正卷积核的参量。

## 交叉熵损失函数

$$E = - \sum p_i \ln Y_i$$



# 误差评估

每次评估误差后根据误差修正卷积核的参量。

## 交叉熵损失函数

$$E = - \sum p_i \ln Y_i$$

其中  $p_i$  是真实概率,  $Y_i$  是 Softmax 函数所得。



9

## 误反向传播



## 输出层-Softmax 函数的梯度

$$\frac{\partial Y_j}{\partial y_i} = \frac{\exp(y_i)(\sum \exp(y_j) - \exp(y_i))}{(\sum \exp(y_j))^2} = Y_i(1 - Y_i) \quad (i = j)$$



## 输出层-Softmax 函数的梯度

$$\frac{\partial Y_j}{\partial y_i} = \frac{\exp(y_i)(\sum \exp(y_j) - \exp(y_i))}{(\sum \exp(y_j))^2} = Y_i(1 - Y_i) \quad (i = j)$$

$$\frac{\partial Y_j}{\partial y_i} = \frac{\exp(y_j)(0 - \exp(y_i))}{(\sum \exp(y_j))^2} = -Y_i \cdot Y_j \quad (i \neq j)$$





## 输出层-Softmax 函数的梯度

$$\frac{\partial Y_j}{\partial y_i} = \frac{\exp(y_i)(\sum \exp(y_j) - \exp(y_i))}{(\sum \exp(y_j))^2} = Y_i(1 - Y_i) \quad (i = j)$$

$$\frac{\partial Y_j}{\partial y_i} = \frac{\exp(y_j)(0 - \exp(y_i))}{(\sum \exp(y_j))^2} = -Y_i \cdot Y_j \quad (i \neq j)$$

$$grad_i = \frac{\partial E}{\partial y_i} = \frac{\partial(\sum E_j)}{\partial y_i} = \sum \frac{\partial E_j}{\partial Y_j} \cdot \frac{\partial Y_j}{\partial y_i} = -\frac{p_i}{Y_i} Y_i(1 - Y_i) + \sum_{j \neq i} -\frac{p_j}{Y_j} Y_j Y_i$$



## 输出层-Softmax 函数的梯度

$$\frac{\partial Y_j}{\partial y_i} = \frac{\exp(y_i)(\sum \exp(y_j) - \exp(y_i))}{(\sum \exp(y_j))^2} = Y_i(1 - Y_i) \quad (i = j)$$

$$\frac{\partial Y_j}{\partial y_i} = \frac{\exp(y_j)(0 - \exp(y_i))}{(\sum \exp(y_j))^2} = -Y_i \cdot Y_j \quad (i \neq j)$$

$$grad_i = \frac{\partial E}{\partial y_i} = \frac{\partial(\sum E_j)}{\partial y_i} = \sum \frac{\partial E_j}{\partial Y_j} \cdot \frac{\partial Y_j}{\partial y_i} = -\frac{p_i}{Y_i} Y_i(1 - Y_i) + \sum_{j \neq i} -\frac{p_j}{Y_j} Y_j Y_i$$

结合  $\sum p_i = 1$ , 可以得到:

$$grad_i = Y_i - p_i$$



## 输出层-全连接神经元的梯度

方便起见，此后我们把上一层（也就是正向传播的下一层）所得梯度成为  $pre_i$



## 输出层-全连接神经元的梯度

方便起见，此后我们把上一层（也就是正向传播的下一层）所得梯度成为  $pre_i$

$$grad_i = \frac{\partial E}{\partial y_i} = \sum_{j=0}^9 \frac{\partial E_j}{\partial y_j} \cdot \frac{\partial y_j}{\partial x_i} = \sum_{j=0}^9 pre_j \cdot w_{ij}$$

其中  $w_{ij}$  为全输出层神经元将  $x_i$  贡献到  $y_j$  的权重。



# 池化层

在全连接层中我们得到了 12 个  $4 \times 4$  矩阵的梯度，那么经过池化层的反向传播后，矩阵扩展为原本的  $8 \times 8$  的大小。



# 池化层

在全连接层中我们得到了 12 个  $4 \times 4$  矩阵的梯度，那么经过池化层的反向传播后，矩阵扩展为原本的  $8 \times 8$  的大小。  
由于这里池化采取的是取  $\max$ ，所以反向传播的时候只要维持好这个性质就可以了。



# 池化层

在全连接层中我们得到了 12 个  $4 \times 4$  矩阵的梯度，那么经过池化层的反向传播后，矩阵扩展为原本的  $8 \times 8$  的大小。

由于这里池化采取的是取  $\max$ ，所以反向传播的时候只要维持好这个性质就可以了。一种可行的构造是维持池化窗口中最大值位置上的数不变，其余位置填上 0。



# 卷积层

显然 Relu 函数的导数为:  $Relu'(x) = [x > 0]$ 。





# 卷积层

显然 Relu 函数的导数为:  $Relu'(x) = [x > 0]$ 。  
结合矩阵卷积的求导方法, 有:

$$grad_i = \sum (pre_i \cdot Relu'(y_i)) * w'_{ij}$$

其中  $w'_{ij}$  为将输入的第  $j$  张图片贡献到输出第  $i$  张图片的权值矩阵的旋转矩阵。



# 卷积层

显然 Relu 函数的导数为:  $Relu'(x) = [x > 0]$ 。  
结合矩阵卷积的求导方法, 有:

$$grad_i = \sum (pre_i \cdot Relu'(y_i)) * w'_{ij}$$

其中  $w'_{ij}$  为将输入的第  $j$  张图片贡献到输出第  $i$  张图片的权值矩阵的旋转矩阵。  
前两层的梯度推导是类似的。



# 参数更新

我们已经求出每一层的输出  $y_i$  关于  $E$  的梯度。



## 参数更新

我们已经求出每一层的输出  $y_i$  关于  $E$  的梯度。鉴于每一层的转换都形如：

$$y_i = w_{ij}x_j + b_i$$



## 参数更新

我们已经求出每一层的输出  $y_i$  关于  $E$  的梯度。鉴于每一层的转换都形如：

$$y_i = w_{ij}x_j + b_i$$

显然有：

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial y_i} \cdot x_j, \quad \frac{\partial E}{\partial b_i} = \frac{\partial E}{\partial y_i}$$



## 参数更新

我们已经求出每一层的输出  $y_i$  关于  $E$  的梯度。鉴于每一层的转换都形如：

$$y_i = w_{ij}x_j + b_i$$

显然有：

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial y_i} \cdot x_j, \quad \frac{\partial E}{\partial b_i} = \frac{\partial E}{\partial y_i}$$

据此我们可以更新任意参量  $X$ ：

$$X' = X - \alpha \cdot \text{grad}_X$$

其中  $\alpha$  为学习率。





#### 四

## 实际训练中的发现

# 训练参数与结果

- 训练于 CPU 上进行





# 训练参数与结果

- 训练于 CPU 上进行
- C++ 实现 (使用 OpenCV 库), 采用双卷积层 (6 核、12 核), MaxPooling, Relu 函数激活



## 训练参数与结果

- 训练于 CPU 上进行
- C++ 实现 (使用 OpenCV 库), 采用双卷积层 (6 核、12 核), MaxPooling, Relu 函数激活
- 学习率: 初始设置为 0.03, 随着训练的进行逐渐递减到 0.001, 避免在训练后期步长过大跳过最优点



## 训练参数与结果

- 训练于 CPU 上进行
- C++ 实现 (使用 OpenCV 库), 采用双卷积层 (6 核、12 核), MaxPooling, Relu 函数激活
- 学习率: 初始设置为 0.03, 随着训练的进行逐渐递减到 0.001, 避免在训练后期步长过大跳过最优点
- 训练集: MNIST 训练集提供了 60000 份样本, 循环训练 5 次。
- 测试集: 10000 份与训练集格式相同的样本



## 训练参数与结果

- 训练于 CPU 上进行
- C++ 实现 (使用 OpenCV 库), 采用双卷积层 (6 核、12 核), MaxPooling, Relu 函数激活
- 学习率: 初始设置为 0.03, 随着训练的进行逐渐递减到 0.001, 避免在训练后期步长过大跳过最优点
- 训练集: MNIST 训练集提供了 60000 份样本, 循环训练 5 次。
- 测试集: 10000 份与训练集格式相同的样本

训练结果: 成功率约为 98.2%



## 相同参数下使用 PyTorch 进行训练

| Epoch | 训练损失            | 训练准确率           | 测试损失            | 测试准确率           | 95% 置信区间宽度     |
|-------|-----------------|-----------------|-----------------|-----------------|----------------|
| 1     | 0.183202        | 94.3783%        | 0.064063        | 97.8500%        | 0.5696%        |
| 2     | 0.053567        | 98.3317%        | 0.046267        | 98.4700%        | 0.4825%        |
| 3     | 0.040147        | 98.7183%        | 0.032459        | 98.9100%        | 0.4087%        |
| 4     | 0.025776        | 99.1967%        | 0.023007        | 99.1900%        | 0.3533%        |
| 5     | 0.020643        | 99.3133%        | 0.023859        | 99.1300%        | 0.3659%        |
| 6     | 0.018518        | 99.4417%        | 0.020841        | 99.3200%        | 0.3243%        |
| 7     | 0.012581        | 99.6200%        | 0.020318        | <b>99.3500%</b> | 0.3172%        |
| 8     | 0.011321        | 99.6250%        | <b>0.020098</b> | 99.3600%        | <b>0.3148%</b> |
| 9     | 0.010463        | 99.6533%        | 0.022053        | 99.2900%        | 0.3312%        |
| 10    | <b>0.007992</b> | <b>99.7667%</b> | 0.019869        | 99.3300%        | 0.3220%        |



## 对模型进行一些调整

| No. | 模型名称                | 准确率 (%)      | 95% 置信区间                | 参数量         |
|-----|---------------------|--------------|-------------------------|-------------|
| 3   | 双卷积层 (8,16)         | 98.66        | [98.42%, 98.87%]        | 52, 138     |
| 4   | <b>双卷积层 (32,64)</b> | <b>99.21</b> | <b>[99.02%, 99.37%]</b> | 421, 642    |
| 5   | 双卷积层 (64,128)       | 99.10        | [98.90%, 99.27%]        | 1, 682, 954 |
| 6   | 单卷积层                | 98.22        | [97.94%, 98.46%]        | 804, 554    |
| 7   | 三卷积层                | 97.79        | [97.48%, 98.06%]        | 93, 962     |
| 8   | 双卷积层 + 平均池化         | 99.01        | [98.80%, 99.19%]        | 421, 642    |
| 9   | 双卷积层 + Sigmoid 激活   | 98.21        | [97.93%, 98.45%]        | 421, 642    |



# Discussion

- 卷积核过多为什么效果没有变好?



## Discussion

- 卷积核过多为什么效果没有变好?

### 过拟合

模型对训练集中的数据识别能力极强，但是把其中的一些噪声和不具一般性的特征作为了“标准”，从而在测试集中表现不佳。





## Discussion

- 卷积核过多为什么效果没有变好?

### 过拟合

模型对训练集中的数据识别能力极强，但是把其中的一些噪声和不具一般性的特征作为了“标准”，从而在测试集中表现不佳。

- 有没有解决办法?



## Discussion

- 卷积核过多为什么效果没有变好?

### 过拟合

模型对训练集中的数据识别能力极强，但是把其中的一些噪声和不具一般性的特征作为了“标准”，从而在测试集中表现不佳。

- 有没有解决办法?

### Dropout

在训练中随机让部分神经元“失活”，从而避免对某个特征的过度攫取



## Discussion

- 卷积核过多为什么效果没有变好?

### 过拟合

模型对训练集中的数据识别能力极强，但是把其中的一些噪声和不具一般性的特征作为了“标准”，从而在测试集中表现不佳。

- 有没有解决办法?

### Dropout

在训练中随机让部分神经元“失活”，从而避免对某个特征的过度攫取

### 数据增强

对训练集做空间变换 (e.g. 旋转  $10^\circ$ , 平移 1 个单位)



## Discussion

- 当卷积层增加到 3 层时，训练效果反而变差了？



## Discussion

- 当卷积层增加到 3 层时，训练效果反而变差了？

### 梯度消失

当层数过深的时候，链式法则的叠加：

$$\frac{\partial E}{\partial x_i} = \sum_j \left( \frac{\partial E}{\partial y_j} \cdot \frac{\partial y_j}{\partial a_L} \cdot \frac{\partial a_L}{\partial z_L} \cdots \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial x_i} \right)$$



## Discussion

- 当卷积层增加到 3 层时，训练效果反而变差了？

### 梯度消失

当层数过深的时候，链式法则的叠加：

$$\frac{\partial E}{\partial x_i} = \sum_j \left( \frac{\partial E}{\partial y_j} \cdot \frac{\partial y_j}{\partial a_L} \cdot \frac{\partial a_L}{\partial z_L} \cdots \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial x_i} \right)$$

使得浅层网络的梯度变得非常小。于是更新过程：

$$w_{\text{new}} = w_{\text{old}} - \alpha \cdot \frac{\partial E}{\partial w}$$

会非常缓慢。这发生在浅层网络中会导致模型对低级特征的学习效果变差。



# Discussion

- 如果我们需要一个深层网络? Try ResNet!



# Discussion

- 如果我们需要一个深层网络？ Try ResNet!

## shortcut

直接将输入特征“跳过”一个或多个层后加到输出上，以解决深度网络训练中的梯度消失和退化问题。





# Discussion

- 如果我们需要一个深层网络？ Try ResNet!

## shortcut

直接将输入特征“跳过”一个或多个层后加到输出上，以解决深度网络训练中的梯度消失和退化问题。

- 为什么对？



# Discussion

- AveragePooling 劣于 MaxPooling?



# Discussion

- AveragePooling 劣于 MaxPooling?

## Recall

每一个卷积核都用于检测某一个特定的特征，卷积的过程相当于是相似度匹配（向量内积）。



# Discussion

- AveragePooling 劣于 MaxPooling?

## Recall

每一个卷积核都用于检测某一个特定的特征，卷积的过程相当于是相似度匹配（向量内积）。

- 高激活值对应于更显著的特征！



# Discussion

- AveragePooling 劣于 MaxPooling?

## Recall

每一个卷积核都用于检测某一个特定的特征，卷积的过程相当于是相似度匹配（向量内积）。

- 高激活值对应于更显著的特征！

## 特征锐度

AveragePooling 将目标的特征与背景进行稀释，而 MaxPooling 只允许最显著的一部分特征通过。



# Discussion

- Sigmoid 作为激活函数不如 Relu?



# Discussion

- Sigmoid 作为激活函数不如 Relu?

## Recall

Sigmoid 函数:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \leq 0.25$$



# Discussion

- Sigmoid 作为激活函数不如 Relu?

## Recall

Sigmoid 函数:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \leq 0.25$$

- 容易导致梯度消失。





## 其他优化策略

- BatchNorm: 将每一层输入的激活值的分布, 强制性地调整到均值为 0、方差为 1 的标准高斯分布。



## 其他优化策略

- BatchNorm: 将每一层输入的激活值的分布, 强制性地调整到均值为 0、方差为 1 的标准高斯分布。
- 梯度裁剪: 限制梯度的范围, 防止梯度爆炸



## 综合各种策略的模型

| Epoch | 训练损失   | 训练准确率 (%) | 测试损失   | 测试准确率 (%)    | 学习率      |
|-------|--------|-----------|--------|--------------|----------|
| 6     | 0.0600 | 98.19     | 0.0203 | 99.38        | 0.000655 |
| 7     | 0.0514 | 98.41     | 0.0163 | 99.46        | 0.000553 |
| 8     | 0.0468 | 98.61     | 0.0171 | 99.50        | 0.000448 |
| 9     | 0.0437 | 98.67     | 0.0160 | 99.50        | 0.000346 |
| 10    | 0.0397 | 98.81     | 0.0134 | 99.59        | 0.000251 |
| 11    | 0.0374 | 98.90     | 0.0117 | <b>99.64</b> | 0.000166 |



## 综合各种策略的模型

| Epoch | 训练损失   | 训练准确率 (%) | 测试损失   | 测试准确率 (%)    | 学习率      |
|-------|--------|-----------|--------|--------------|----------|
| 6     | 0.0600 | 98.19     | 0.0203 | 99.38        | 0.000655 |
| 7     | 0.0514 | 98.41     | 0.0163 | 99.46        | 0.000553 |
| 8     | 0.0468 | 98.61     | 0.0171 | 99.50        | 0.000448 |
| 9     | 0.0437 | 98.67     | 0.0160 | 99.50        | 0.000346 |
| 10    | 0.0397 | 98.81     | 0.0134 | 99.59        | 0.000251 |
| 11    | 0.0374 | 98.90     | 0.0117 | <b>99.64</b> | 0.000166 |

- 人类的准确率约为 99.70% 到99.80%



## 综合各种策略的模型

| Epoch | 训练损失   | 训练准确率 (%) | 测试损失   | 测试准确率 (%)    | 学习率      |
|-------|--------|-----------|--------|--------------|----------|
| 6     | 0.0600 | 98.19     | 0.0203 | 99.38        | 0.000655 |
| 7     | 0.0514 | 98.41     | 0.0163 | 99.46        | 0.000553 |
| 8     | 0.0468 | 98.61     | 0.0171 | 99.50        | 0.000448 |
| 9     | 0.0437 | 98.67     | 0.0160 | 99.50        | 0.000346 |
| 10    | 0.0397 | 98.81     | 0.0134 | 99.59        | 0.000251 |
| 11    | 0.0374 | 98.90     | 0.0117 | <b>99.64</b> | 0.000166 |

- 人类的准确率约为 99.70% 到99.80%
- 主流模型的训练极限在 99.80% 到99.90%,



## 参考资料

## Reference:

- N. Srivastava et al., "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," J. Mach. Learn. Res., vol. 15, no. 1, pp. 1929–1958, 2014.



## Reference:

- N. Srivastava et al., "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," J. Mach. Learn. Res., vol. 15, no. 1, pp. 1929–1958, 2014.
- K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," arXiv e-prints, vol. abs/1512.03385, 2015.





## Reference:

- N. Srivastava et al., "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," J. Mach. Learn. Res., vol. 15, no. 1, pp. 1929–1958, 2014.
- K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," arXiv e-prints, vol. abs/1512.03385, 2015.
- <https://zhuanlan.zhihu.com/p/122559611>



## Reference:

- N. Srivastava et al., "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," J. Mach. Learn. Res., vol. 15, no. 1, pp. 1929–1958, 2014.
- K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," arXiv e-prints, vol. abs/1512.03385, 2015.
- <https://zhuanlan.zhihu.com/p/122559611>
- weijiakai's blog (C++/OpenCV 实现 CNN 系列)





谢谢